Guía n°

RabbitMQ Cluster con Docker Compose

ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

Subdirección de Diseño
Curricular e Instruccional
202

Guía n°

Nombre de la guía

Sigla	Asignatura	Experiencia de Aprendizaje
Tiempo	**Modalidad de Trabajo**	**Indicadores de Logro**

Contexto

Un cluster de RabbitMQ es un grupo de nodos que comparten el mismo estado, usuarios, colas, exchanges y configuraciones en tiempo de ejecución. Los nodos pueden ser agregados o removidos dinámicamente.

Ventajas de un Cluster:

- Tolerancia a Fallos: Si un nodo cae, los otros continúan operando
- Replicación Automática: Las colas se replican entre nodos
- Load Balancing: Las conexiones de clientes se distribuyen entre nodos
- Escalabilidad Horizontal: Agregar más nodos aumenta la capacidad del sistema

Conceptos Clave

- Cookie de Erlang: String secreto que comparten los nodos para reconocerse como parte del cluster (almacenado en /var/lib/rabbitmq/.erlang.cookie)

- Nombres de Nodo: Cada nodo tiene identificador único en forma rabbit@hostname (ej: rabbit@rabbitmq-1, rabbit@rabbitmq-2)

- Tipos de Nodos:
  - Disc Node (por defecto): Guarda configuración en disco. Debe existir al menos uno
  - Ram Node (obsoleto desde 3.7): Almacena todo en memoria. Ya no se recomienda

Comenzamos

Docker Compose Configuration

```
version: '3.9'

services:
  rabbitmq-1:
    image: rabbitmq:3.12-management-alpine
    container_name: rabbitmq-1
    hostname: rabbitmq-1
    ports:
      - "5672:5672"     # AMQP port
      - "15672:15672"   # Management UI
    environment:
      RABBITMQ_DEFAULT_USER: admin
      RABBITMQ_DEFAULT_PASS: admin123
      RABBITMQ_ERLANG_COOKIE: "secret_cookie_value"
    volumes:
      - rabbitmq_data_1:/var/lib/rabbitmq
      - ./rabbitmq.conf:/etc/rabbitmq/rabbitmq.conf:ro
    networks:
      - rabbitmq_cluster

  rabbitmq-2:
```

```
    image: rabbitmq:3.12-management-alpine
    container_name: rabbitmq-2
    hostname: rabbitmq-2
    ports:
      - "5673:5672"      # AMQP port (mapeado a 5673 en host)
      - "15673:15672"    # Management UI
    environment:
      RABBITMQ_DEFAULT_USER: admin
      RABBITMQ_DEFAULT_PASS: admin123
      RABBITMQ_ERLANG_COOKIE: "secret_cookie_value"  # MISMO COOKIE
    volumes:
      - rabbitmq_data_2:/var/lib/rabbitmq
      - ./rabbitmq.conf:/etc/rabbitmq/rabbitmq.conf:ro
    networks:
      - rabbitmq_cluster
    depends_on:
      - rabbitmq-1

volumes:
  rabbitmq_data_1:
  rabbitmq_data_2:

networks:
  rabbitmq_cluster:
    driver: bridge
```

Archivo de Configuración (rabbitmq.conf)

#Habilitar management plugin

management.load_definitions = /etc/rabbitmq/definitions.json

#Configuración de clustering automático

cluster_formation.peer_discovery_backend = rabbit_peer_discovery_classic_config
cluster_formation.classic_config.nodes.1 = rabbit@rabbitmq-1
cluster_formation.classic_config.nodes.2 = rabbit@rabbitmq-2

#Tiempo de espera para que los nodos se descubran

cluster_formation.node_cleanup.interval = 10000

#Política de sincronización de colas (todos los nodos)

queue_master_location = all

Levantar el Cluster

Crear directorio del proyecto
mkdir rabbitmq-avanzado
cd rabbitmq-avanzado

Levantar el cluster
docker-compose up -d

Verificar que ambos nodos están corriendo
docker-compose ps

Ver logs
docker-compose logs rabbitmq-1
docker-compose logs rabbitmq-2

Acceder a Management UI
Nodo 1: http://localhost:15672 (admin/admin123)
Nodo 2: http://localhost:15673 (admin/admin123)

Verificar el Cluster

```
# Conectarse al nodo 1
docker exec rabbitmq-1 rabbitmqctl cluster_status

# Salida esperada:
# Cluster status of node rabbit@rabbitmq-1 ...
# [{nodes,[{disc,[rabbit@rabbitmq-1,rabbit@rabbitmq-2]}]},
#  {running_nodes,[rabbit@rabbitmq-1,rabbit@rabbitmq-2]},
```

Crear Proyecto Spring Boot

PASO 1: Generar Proyecto con Spring Initializr

Opción A: Usar la Interfaz Web (Recomendado)

1. Ve a: https://start.spring.io/
2. Rellena los campos con estos valores:

Project: Maven Project
Language: Java
Spring Boot: 3.2.x (o superior)
Group: com.example
Artifact: rabbitmq-avanzado
Packaging: Jar
Java: 17

3. En la sección "Dependencies" agrega estas 3 dependencias:
   a. Spring Web
   b. Spring for RabbitMQ (búscalo como "AMQP")
   c. Lombok
4. Click en GENERATE
5. Descomprime el archivo descargado en tu carpeta rabbitmq-avanzado

Opción B: Usar Maven CLI

```
mvn archetype:generate \
 -DgroupId=com.example \
```

```
-DartifactId=rabbitmq-avanzado \
-DarchetypeArtifactId=maven-archetype-quickstart \
-DinteractiveMode=false
```

Crear Clase Principal

Archivo: src/main/java/com/example/RabbitMQAdvancedApplication.java

```
package com.example;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.scheduling.annotation.EnableScheduling;

@SpringBootApplication
@EnableScheduling  // Para el Circuit Breaker (@Scheduled)
public class RabbitMQAdvancedApplication {

   public static void main(String[] args) {
      SpringApplication.run(RabbitMQAdvancedApplication.class, args);
   }
}
```

Configurar application.properties

Archivo: src/main/resources/application.properties

```
# ==========================================
# CONFIGURACIÓN DE LA APLICACIÓN
# ==========================================
spring.application.name=rabbitmq-avanzado
server.port=8080

# ==========================================
# CONFIGURACIÓN DE RABBITMQ
# ==========================================
spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=admin
```

```
spring.rabbitmq.password=admin123
spring.rabbitmq.virtual-host=/

# Configuración de listeners
spring.rabbitmq.listener.simple.prefetch=1
spring.rabbitmq.listener.simple.concurrency=3
spring.rabbitmq.listener.simple.max-concurrency=5

# ==========================================
# CONFIGURACIÓN DE LOGGING
# ==========================================
logging.level.root=INFO
logging.level.com.example=DEBUG
logging.level.org.springframework.amqp=DEBUG
logging.pattern.console=%d{HH:mm:ss.SSS} [%thread] %-5level %logger{36} -
%msg%n
```

Verificar pom.xml

Tu pom.xml debe tener estas dependencias principales:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
      http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.2.0</version>
    <relativePath/>
  </parent>

  <groupId>com.example</groupId>
  <artifactId>rabbitmq-avanzado</artifactId>
  <version>1.0.0</version>
  <name>RabbitMQ Avanzado</name>

  <properties>
```

```
    <java.version>17</java.version>
  </properties>

  <dependencies>
    <!-- Spring Web -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <!-- Spring AMQP (RabbitMQ) -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-amqp</artifactId>
    </dependency>

    <!-- Lombok -->
    <dependency>
      <groupId>org.projectlombok</groupId>
      <artifactId>lombok</artifactId>
      <optional>true</optional>
    </dependency>

    <!-- Testing -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-test</artifactId>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
      </plugin>
    </plugins>
  </build>

</project>
```

Compilar y Ejecutar

En la raíz del proyecto (donde está pom.xml)

1. Compilar el proyecto
mvn clean install

2. Ejecutar Spring Boot
mvn spring-boot:run

O si usas un IDE (IntelliJ, VSCode):
Click derecho en RabbitMQAdvancedApplication.java → Run

Deberías ver en los logs:

Started RabbitMQAdvancedApplication in X.XXX seconds (JVM running for X.XXX)

Gestión Programática de Recursos

¿Qué es RabbitAdmin?

RabbitAdmin es una clase de Spring AMQP que permite gestionar recursos de RabbitMQ (colas, exchanges, bindings) de forma programática en tiempo de ejecución, sin depender de @Bean en la clase de configuración.

Casos de Uso:

- Crear colas dinámicamente cuando llega un cliente nuevo.
- Eliminar colas cuando un cliente se desconecta.
- Modificar bindings según reglas de negocio.
- Auditar y monitorear el estado de recursos.

Configuración de RabbitAdmin

Archivo: src/main/java/com/example/config/RabbitAdminConfig.java

```
package com.example.config;

import org.springframework.amqp.rabbit.core.RabbitAdmin;
import org.springframework.amqp.rabbit.core.RabbitTemplate;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.amqp.core.ConnectionFactory;

@Configuration
public class RabbitAdminConfig {

    @Bean
    public RabbitAdmin rabbitAdmin(ConnectionFactory connectionFactory) {
        return new RabbitAdmin(connectionFactory);
    }

    @Bean
    public RabbitTemplate rabbitTemplate(ConnectionFactory connectionFactory) {
        return new RabbitTemplate(connectionFactory);
    }
}
```

Servicio de Gestión (RabbitResourceManager)

Archivo: src/main/java/com/example/service/RabbitResourceManager.java

```
package com.example.service;

import org.springframework.amqp.rabbit.core.RabbitAdmin;

import org.springframework.amqp.core.*;

import org.springframework.stereotype.Service;

import lombok.RequiredArgsConstructor;

import lombok.extern.slf4j.Slf4j;


@Service

@Slf4j

@RequiredArgsConstructor

public class RabbitResourceManager {

private final RabbitAdmin rabbitAdmin;

public void createQueue(String queueName) {

try {

Queue queue = QueueBuilder.durable(queueName).build();

rabbitAdmin.declareQueue(queue);

log.info("✓ Cola creada: {}", queueName);

} catch (Exception e) {

log.error("✗ Error al crear cola {}: {}", queueName, e.getMessage());

}
```

```
}
public void createExchange(String exchangeName) {
try {
Exchange exchange = new DirectExchange(exchangeName, true, false);
rabbitAdmin.declareExchange(exchange);
log.info("✓ Exchange creado: {}", exchangeName);
} catch (Exception e) {
log.error("✗ Error al crear exchange {}: {}", exchangeName, e.getMessage());


}
}
public void createBinding(String queueName, String exchangeName, String routingKey) {
try {
Binding binding = BindingBuilder
.bind(new Queue(queueName))
.to(new DirectExchange(exchangeName))
.with(routingKey);
rabbitAdmin.declareBinding(binding);
log.info("✓ Binding creado: {} -> {}", queueName, exchangeName);
} catch (Exception e) {
log.error("✗ Error al crear binding: {}", e.getMessage());
}
}
public QueueInformation getQueueInfo(String queueName) {
```

```
try {
QueueInformation info = rabbitAdmin.getQueueInfo(queueName);
if (info != null) {
log.info("Cola: {}, Mensajes: {}, Consumidores: {}",
info.getName(), info.getMessageCount(), info.getConsumerCount());
return info;
}
return null;
} catch (Exception e) {
log.error("Error al obtener info de cola {}: {}", queueName, e.getMessage());
return null;
}
}
public void deleteQueue(String queueName) {
try {
rabbitAdmin.deleteQueue(queueName);
log.info("✓ Cola eliminada: {}", queueName);
} catch (Exception e) {
log.error("✗ Error al eliminar cola {}: {}", queueName, e.getMessage());
}
}
public void deleteExchange(String exchangeName) {
try {
rabbitAdmin.deleteExchange(exchangeName);
log.info("✓ Exchange eliminado: {}", exchangeName);
```

```
} catch (Exception e) {
log.error("✗ Error al eliminar exchange {}: {}", exchangeName, e.getMessage());
}
}
public void purgeQueue(String queueName) {
try {
int purged = rabbitAdmin.purgeQueue(queueName);
log.info("✓ Cola purgada: {}, mensajes eliminados: {}", queueName, purged);
} catch (Exception e) {
log.error("✗ Error al purgar cola {}: {}", queueName, e.getMessage());
}
}
}
```

Controller para Exponer Endpoints

Archivo: src/main/java/com/example/controller/RabbitMQController.java

```
package com.example.controller;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import com.example.service.RabbitResourceManager;
import lombok.RequiredArgsConstructor;
@RestController
@RequestMapping("/api/rabbitmq")
```

```
@RequiredArgsConstructor
public class RabbitMQController {
    private final RabbitResourceManager resourceManager;

    @PostMapping("/queues")
    public ResponseEntity<String> createQueue(@RequestParam String queueName) {
        resourceManager.createQueue(queueName);
        return ResponseEntity.status(HttpStatus.CREATED)
            .body("Cola '" + queueName + "' creada exitosamente");
    }
    @PostMapping("/exchanges")
    public ResponseEntity<String> createExchange(@RequestParam String exchangeName) {
        resourceManager.createExchange(exchangeName);
        return ResponseEntity.status(HttpStatus.CREATED)
            .body("Exchange '" + exchangeName + "' creado exitosamente");
    }
    @PostMapping("/bindings")
    public ResponseEntity<String> createBinding(
            @RequestParam String queueName,
            @RequestParam String exchangeName,
            @RequestParam String routingKey) {
        resourceManager.createBinding(queueName, exchangeName, routingKey);
        return ResponseEntity.status(HttpStatus.CREATED)
            .body("Binding creado: " + queueName + " <-> " + exchangeName);
    }
```

```
    @GetMapping("/queues/{queueName}")
    public ResponseEntity<?> getQueueInfo(@PathVariable String queueName) {
        var info = resourceManager.getQueueInfo(queueName);
        return info != null ? ResponseEntity.ok(info) : ResponseEntity.notFound().build();
    }
    @DeleteMapping("/queues/{queueName}")
    public ResponseEntity<String> deleteQueue(@PathVariable String queueName) {
        resourceManager.deleteQueue(queueName);
        return ResponseEntity.ok("Cola '" + queueName + "' eliminada");
    }
    @PostMapping("/queues/{queueName}/purge")
    public ResponseEntity<String> purgeQueue(@PathVariable String queueName) {
        resourceManager.purgeQueue(queueName);
        return ResponseEntity.ok("Cola '" + queueName + "' purgada");
    }
}
```

Control Dinámico de Listeners

¿Qué es un Listener Container?

Un Listener Container es el componente de Spring AMQP que mantiene una conexión abierta a RabbitMQ y escucha mensajes continuamente.

Métodos Clave:

- pause() → Pausa el consumo (conexión permanece abierta)
- resume() → Reanuda el consumo
- stop() → Detiene completamente el listener
- start() → Inicia el listener

Configuración de Listeners

Archivo: src/main/java/com/example/config/RabbitListenerConfig.java

```
package com.example.config;

import
org.springframework.amqp.rabbit.listener.SimpleRabbitListenerContainerFactory;
import org.springframework.amqp.rabbit.connection.ConnectionFactory;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class RabbitListenerConfig {

  @Bean(name = "orderListenerFactory")
  public SimpleRabbitListenerContainerFactory orderListenerFactory(
      ConnectionFactory connectionFactory) {
    SimpleRabbitListenerContainerFactory factory = new
SimpleRabbitListenerContainerFactory();
    factory.setConnectionFactory(connectionFactory);
    factory.setConcurrentConsumers(3);
    factory.setMaxConcurrentConsumers(5);
    return factory;
  }

  @Bean(name = "paymentListenerFactory")
  public SimpleRabbitListenerContainerFactory paymentListenerFactory(
      ConnectionFactory connectionFactory) {
    SimpleRabbitListenerContainerFactory factory = new
SimpleRabbitListenerContainerFactory();
    factory.setConnectionFactory(connectionFactory);
    factory.setConcurrentConsumers(1);
    return factory;
```

```
    }
}
```

Listeners de Ejemplo

Archivo: src/main/java/com/example/listener/OrderListener.java

```
package com.example.listener;


import org.springframework.amqp.rabbit.annotation.RabbitListener;

import org.springframework.stereotype.Service;

import lombok.extern.slf4j.Slf4j;


@Service

@Slf4j

public class OrderListener {


    @RabbitListener(

        id = "order-listener",

        queues = "orders.queue",

        containerFactory = "orderListenerFactory"

    )

    public void processOrder(String orderId) {
```

```
        try {
            log.info(" Procesando orden: {}", orderId);
            Thread.sleep(2000);
            log.info(" Orden procesada: {}", orderId);
        } catch (InterruptedException e) {
            log.error("Error: {}", orderId, e);
            Thread.currentThread().interrupt();
        }
    }
}
```

Servicio de Gestión de Listeners

Archivo: src/main/java/com/example/service/ListenerManagementService.java

```
package com.example.service;

import org.springframework.rabbit.listener.MessageListenerContainer;
import org.springframework.rabbit.listener.RabbitListenerEndpointRegistry;
import org.springframework.stereotype.Service;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import java.util.List;

@Service
@Slf4j
@RequiredArgsConstructor
```

```
public class ListenerManagementService {
    private final RabbitListenerEndpointRegistry registry;
    public void pauseListener(String listenerId) {
        try {
            MessageListenerContainer container = registry.getListenerContainer(listenerId);
            if (container != null && container.isRunning()) {
                container.pause();
                log.info("  Listener pausado: {}", listenerId);
            }
        } catch (Exception e) {
            log.error("Error al pausar listener: {}", e.getMessage());
        }
    }

    public void resumeListener(String listenerId) {
        try {
            MessageListenerContainer container = registry.getListenerContainer(listenerId);
            if (container != null) {
                container.resume();
                log.info("  Listener reanudado: {}", listenerId);
            }
        } catch (Exception e) {
            log.error("Error al reanudar listener: {}", e.getMessage());
        }
    }
```

```
    public List<String> getAllListenerIds() {
        return registry.getListenerContainers()
            .stream()
            .map(c -> (String) c.getListenerId())
            .toList();
    }
}
```

Controller para Controlar Listeners

Archivo: src/main/java/com/example/controller/ListenerController.java

```
package com.example.controller;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import com.example.service.ListenerManagementService;
import lombok.RequiredArgsConstructor;

@RestController
@RequestMapping("/api/listeners")
@RequiredArgsConstructor
public class ListenerController {

    private final ListenerManagementService listenerService;

    @GetMapping
```

```
    public ResponseEntity<Object> getAllListeners() {
        return ResponseEntity.ok(listenerService.getAllListenerIds());
    }
    @PostMapping("/{listenerId}/pause")
    public ResponseEntity<String> pauseListener(@PathVariable String listenerId) {
        listenerService.pauseListener(listenerId);
        return ResponseEntity.ok("Listener pausado: " + listenerId);
    }
    @PostMapping("/{listenerId}/resume")
    public ResponseEntity<String> resumeListener(@PathVariable String listenerId) {
        listenerService.resumeListener(listenerId);
        return ResponseEntity.ok("Listener reanudado: " + listenerId);
    }
}
```

Testing

Asegurate que todo esté corriendo

#Terminal : Cluster de RabbitMQ

docker-compose up -d

#Verificar

docker-compose ps

#Terminal : Spring Boot

mvn spring-boot:run

#Resultado esperado:

#Started RabbitMQAdvancedApplication in X.XXX seconds

Endpoints para Testing

Crear una cola

curl -X POST **"http://localhost:8080/api/rabbitmq/queues?queueName=orders.queue"**

Ver listeners

curl http://localhost:8080/api/listeners

Pausar listener

curl -X POST **"http://localhost:8080/api/listeners/order-listener/pause"**

Reanudar listener

curl -X POST **"http://localhost:8080/api/listeners/order-listener/resume"**

Verificar en Management UI

Nodo 1: http://localhost:15672 (admin/admin123)
Nodo 2: http://localhost:15673 (admin/admin123)

Validar:

- ✅ Colas aparecen en ambos nodos
- ✅ Exchanges replicados
- ✅ Bindings correctos
- ✅ Mensajes distribuidos